

A background image of a city skyline at sunset. The sky is a mix of dark blue, purple, and orange. Silhouettes of buildings and construction cranes are visible against the horizon.

Design Patterns: Introduction & Structural

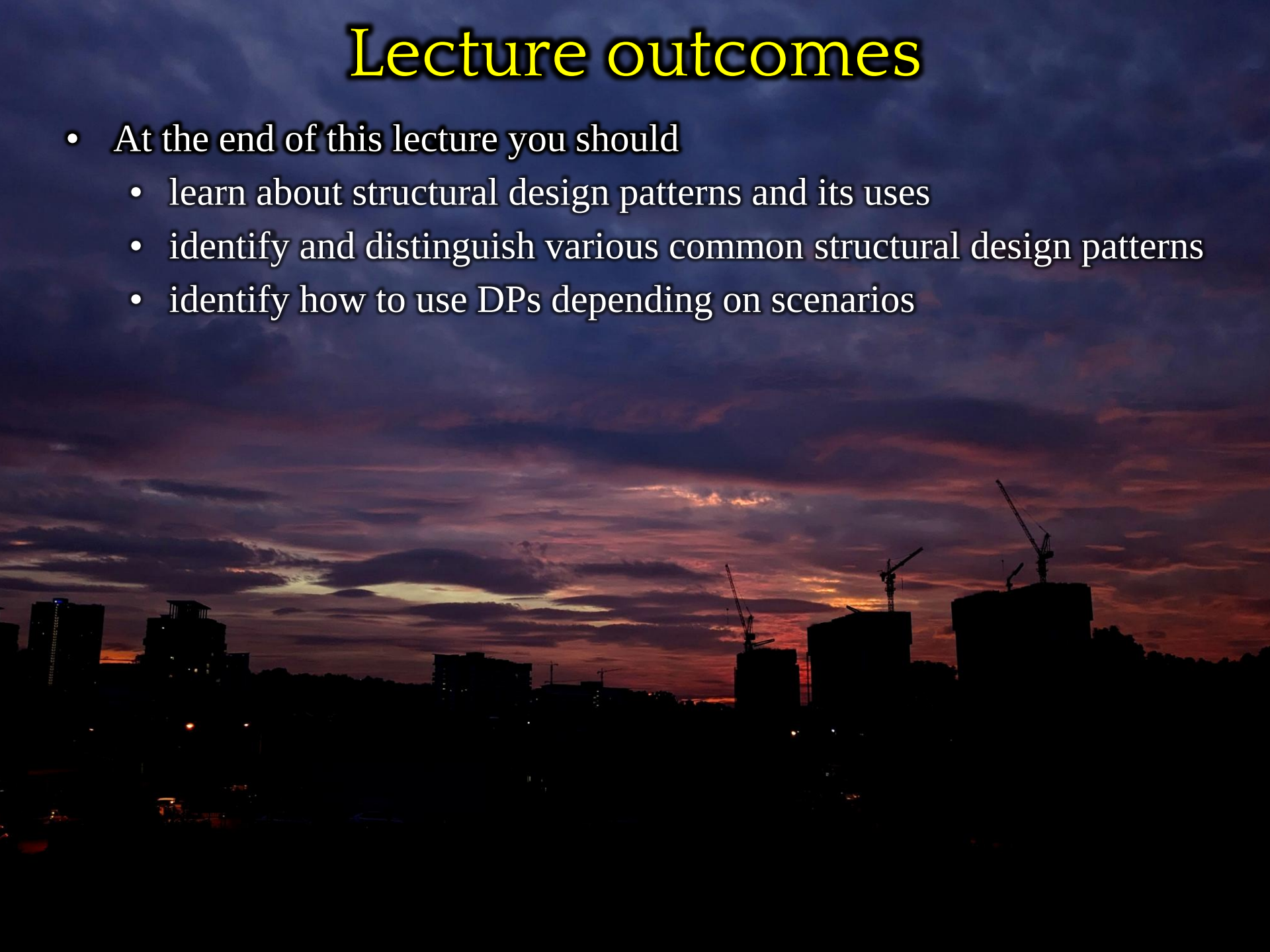
CCP6224

Object-oriented Analysis and Design

Portions of this file have content that were adapted from [1] Object-Oriented Software Engineering Using UML, Patterns, and Java, [2] OOSE Practical software development using UML and Java, [3] <http://kremer.cpsc.ucalgary.ca/patterns/index.html>, [4] <http://www.tutorialspoint.com/index.htm>, [5] <http://www.avajava.com/>, [6] <https://dzone.com>, [7] https://sourcemaking.com/design_patterns/structural_patterns, [8] <http://uosis.mif.vu.lt/~plukas/resources/DPBook/excerpts/patterns.htm> and [9] <http://www.blackwasp.co.uk/gofpatterns.aspx>

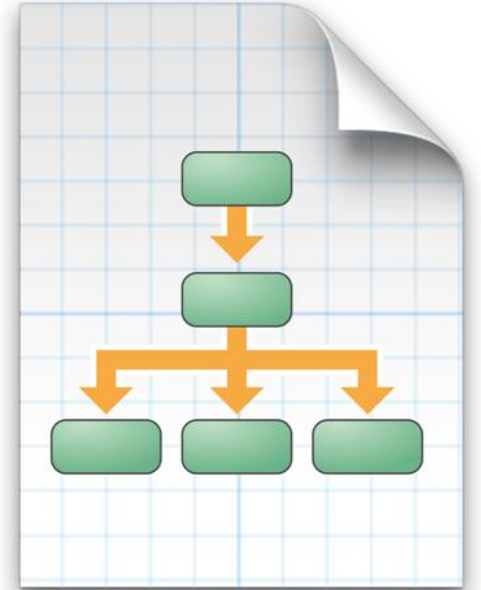
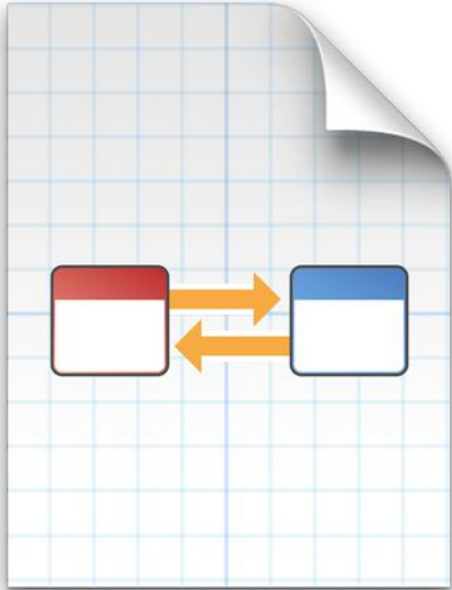
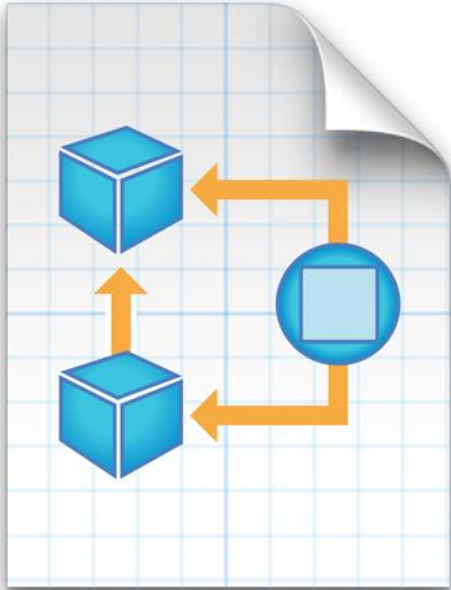
Lecture outcomes

- At the end of this lecture you should
 - learn about structural design patterns and its uses
 - identify and distinguish various common structural design patterns
 - identify how to use DPs depending on scenarios



Motivation

- Developing software from scratch is hard – developing reusable software is even harder
- To help, patterns and frameworks are used because they are proven solutions



Reusability and Reuse

- Reuse and design for reusability should be part of the culture of software development organizations
- But there are problems to overcome:
 - Why take the extra time needed to develop something that will benefit other projects/customers?
 - Management may only reward the efforts of people who create the visible ‘final products’.
 - Reusable software is often created in a hurry and without enough attention to quality.

“Developers tend not develop high quality reusable components, so there is often little to reuse”

Encouraging reuse

- To solve the problem, recognize that:
 - This vicious cycle costs money
 - Investment in reusable code is important
 - Attention to quality of reusable components is essential
 - So that potential reusers have confidence in them
 - The quality of a software product is only as good as its lowest-quality reusable component
 - Developing reusable components can often simplify design
- Reuse is emphasised in OOP with the use of frameworks and design patterns

Reuse-based Software Engineering

- There is a need for reusable and flexible designs to achieve better software, more quickly and at lower cost
 - **Application system reuse**
The whole of an application system may be reused either by incorporating it without change into other systems (COTS reuse) or by configuring the application for different clients.
 - **Component and class reuse**
Components of an application from sub-systems to single classes may be reused.
 - **Object and function reuse**
Software components that implement a single well-defined object or function may be reused

Reusing OO class objects

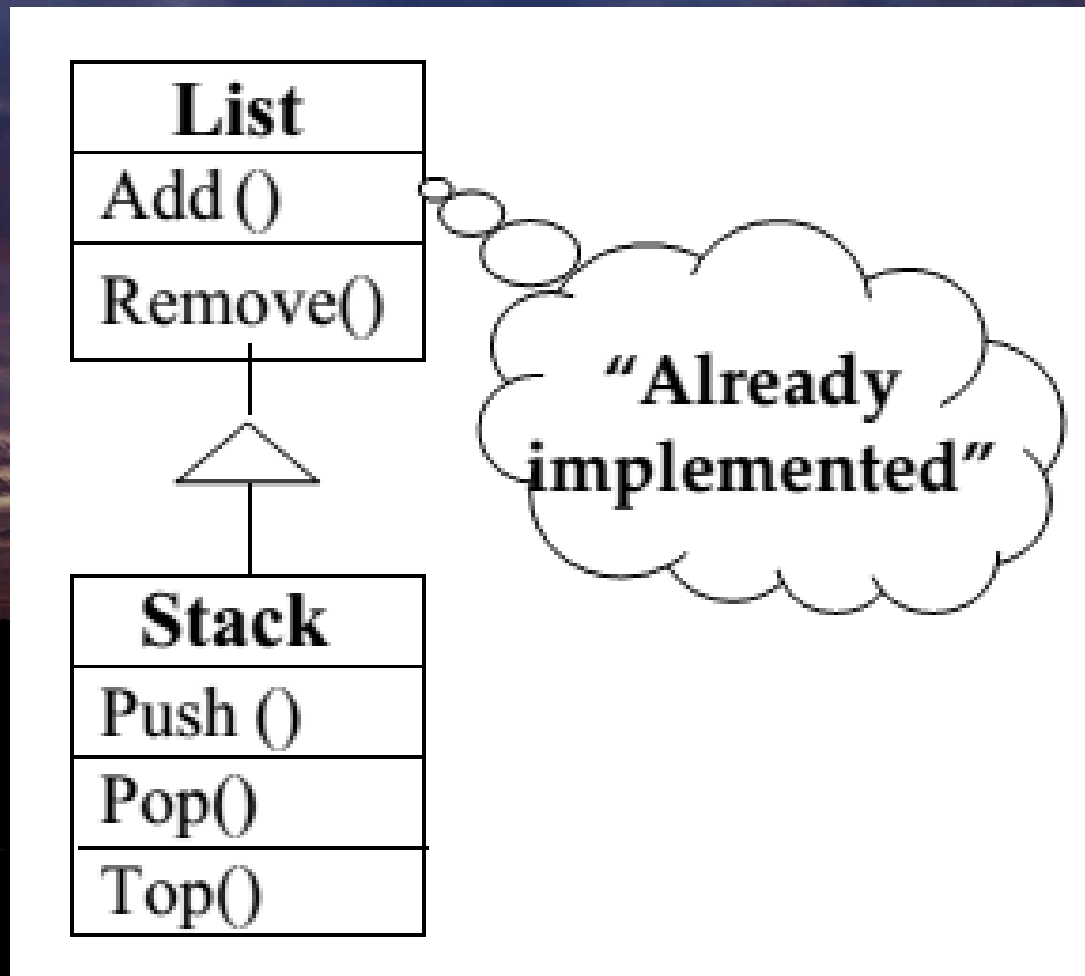
- Look for existing classes in class libraries
- Select data structures appropriate to the algorithms
 - Container classes
 - Arrays, lists, queues, stacks, sets, trees, ...
- Adjust the class (libraries)
 - Change the API if you have the source code.
 - Use adapter objects if you don't have access
- Object-oriented reuse on a class-level is often too fine-grained and too specialized
- Object-oriented reuse is best supported through larger-grain abstractions (frameworks/patterns)

The problem

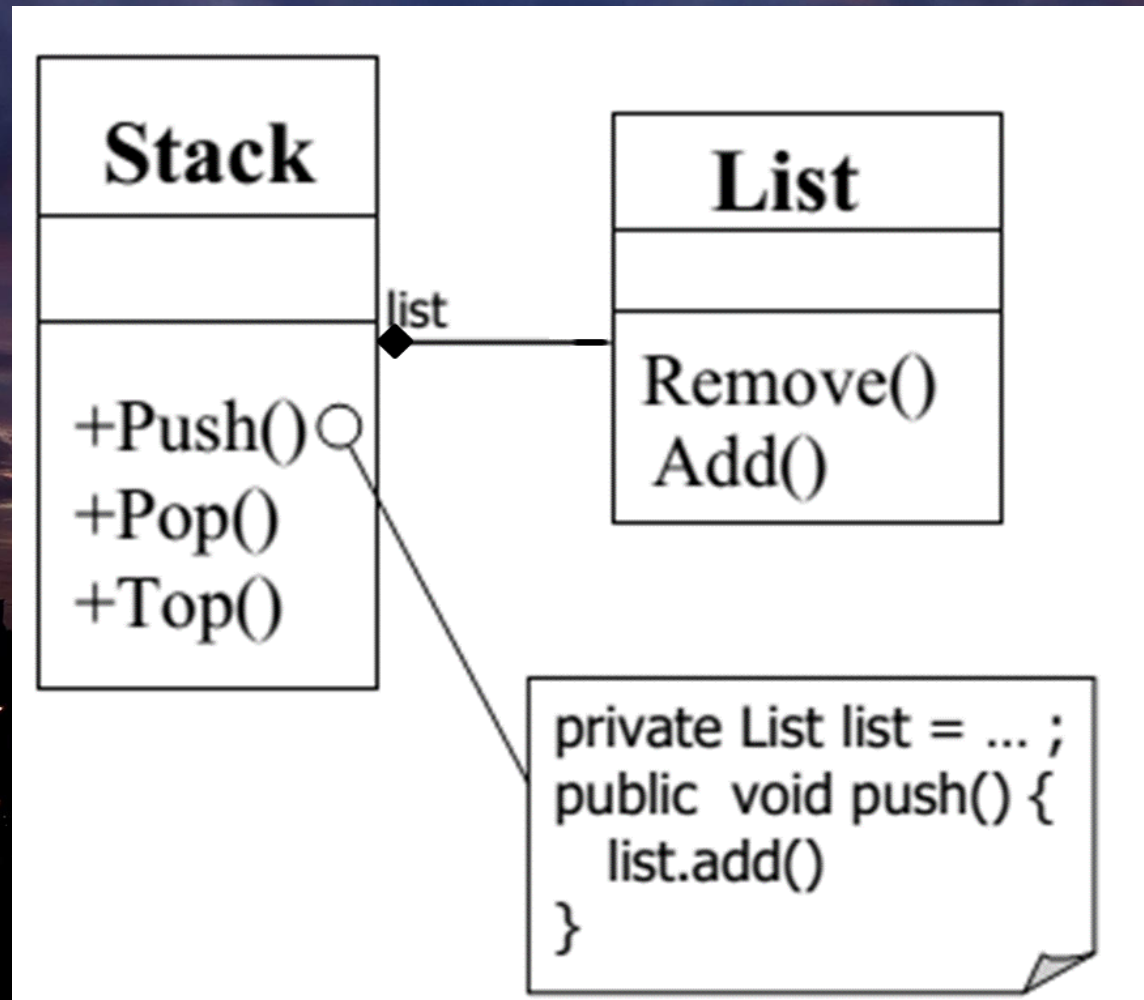
- Design is subjective and there is no sure-fire way of determining the best design short of actually implementing and comparing multiple designs
- Example.
 - You have code to create a 'List' object made with node objects
 - Later you realise you need a 'Stack' type object
 - Various methods exist to realise the 'Stack' but which is the best?



- Do you create the Stack as a implementation inheritance? (generalization method where you inherit from an existing class)
- But what would happen if the 'Stack' object calls **Remove ()** instead of **Pop ()**?



- What about creating the Stack as a delegation instead? (composition method)
- A Stack would then make use of List node objects but rewrite the Pop and Push methods to incorporate Remove and Add



Comparison

Delegation

Pros:

Flexibility → Any delegate object can be replaced at run time by another one (as long as it has the same type)
Details of the delegate can be hidden
Low coupling from client to delegate class

Cons:

Inefficiency: Many objects available, flat hierarchy
Highly parameterized → difficult to understand, risk of having inconsistencies in the architecture

Inheritance

Pros:

Straightforward to use
Static hierarchy at compile time ensures consistent architecture
Easy to implement (override) new functionality
Good for creating and maintaining different implementations according to a common interface

Cons:

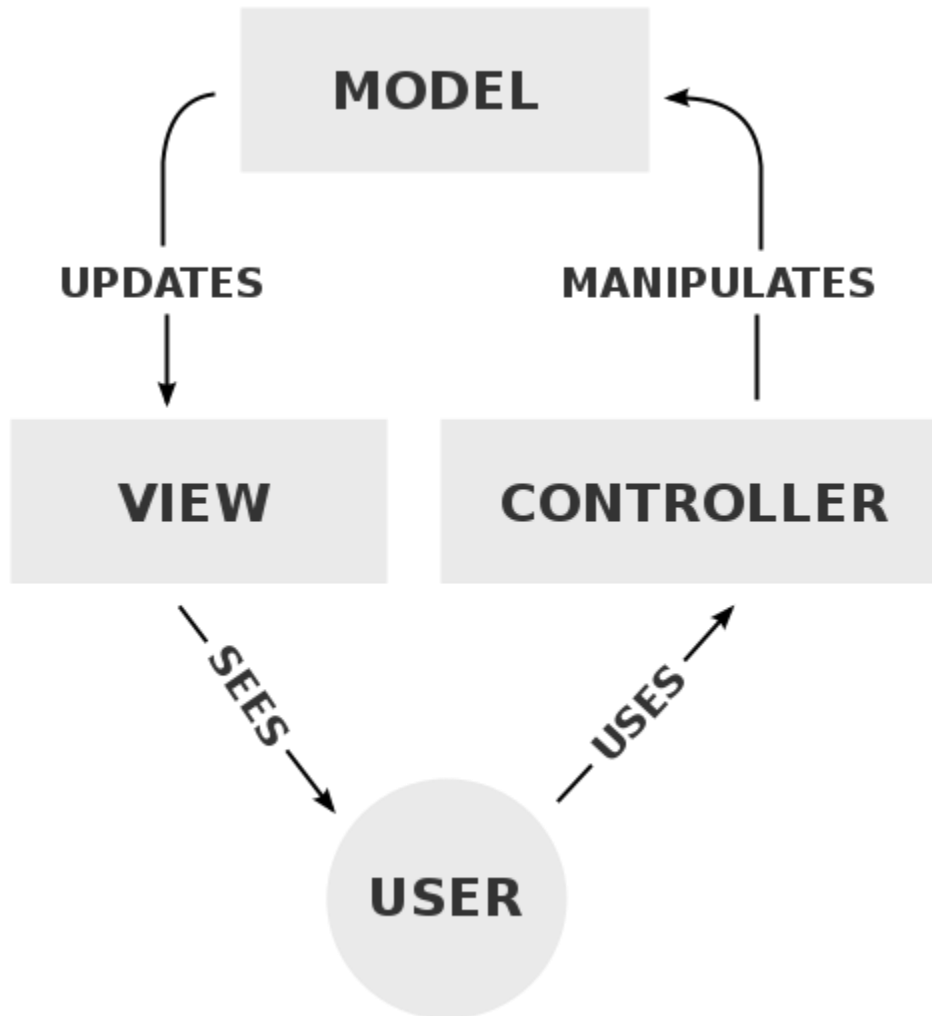
Inheritance exposes (undesired) details of the parent class
High coupling between classes
Inheritance hierarchy cannot change at runtime
Substitution of the subclasses for a super class in any of the client code possible → may lead to unwanted conditions

Resolving the debate

- Although there still is no guaranteed way to promise a successful design, there are several tried-and-tested design patterns (solutions) intended for common problem scenarios.
- A design pattern is a template solution that developers have refined over time to solve a range of recurring problems
- Design patterns are sufficiently abstract to be reused in different settings.
- Makes software modifiable and extensible to minimize the cost of future changes
- Patterns merely rely on object characteristics such as:
 - inheritance
 - delegation
 - dynamic binding

Model-View-Controller

- Basic pattern for GUIs



Model-View-Controller Design Pattern

- Foundational design pattern for Graphical User Interfaces. It helps make GUI-based programs more reusable and avoid bugs.
 - Separate GUI code from model code.
 - Replacing the GUI classes should not need to change anything in the model's classes.
 - Violated when GUI classes contain anything that should be in model's classes.
- <https://www.geeksforgeeks.org/mvc-design-pattern/>
- For more information, Google “Model-View-Controller Design Pattern”



Gang of Four

- The *Gang of Four* are the four authors of the book, "Design Patterns: Elements of Reusable Object-Oriented Software" → Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides
- They describe 23 typical design patterns with sample UML diagrams, source code and real-world examples for each
- These 23 DPs were roughly categorised into the following:
 - **Creational patterns** provide ways to instantiate single objects or groups of related objects → *Concerned with the process of object creation*
 - **Structural patterns** provide a manner to define relationships between classes or objects → *Deal with the composition of classes of objects towards higher level constructs*
 - **Behavioural patterns** define manners of communication between classes and objects → *Characterizes the ways in which classes or objects interact and distribute responsibility*

The Gang Of Four



December 1999



Design pattern categories

Creational	Structural	Behavioural
Factory Method Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Flyweight Façade Proxy	Interpreter Template Method Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Design Pattern content

- A typical design pattern has
 - **Name** - A meaningful pattern identifier
 - **Intent** - What does the pattern do? What is its rationale?
 - **Motivation** (Problem Description) - A scenario that illustrates a design problem and how the class structures in the pattern solve the problem
 - **Applicability** - What are situations in which the pattern can be applied?
 - **Structure** (Solution) - Description of the pattern by means of standardized representation techniques (class diagrams, UML...)
 - **Consequences** - The results and trade-offs of applying the pattern.
 - **Sample Code/Examples**



COMMON DESIGN PATTERNS

(Composite - Structural)

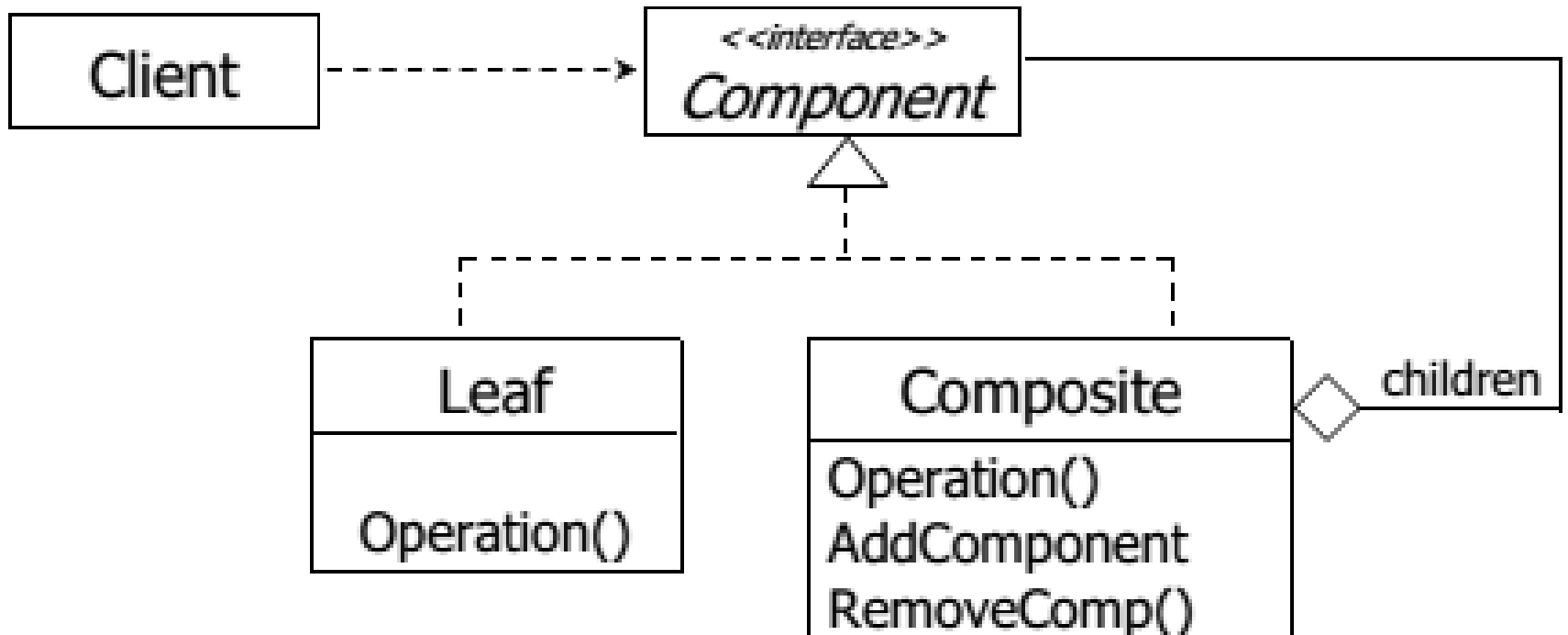
Composite DP scenario

- You write your own program code library to draw objects on the screen.
 - The library will contain items like
 - basic lines i.e. two coordinate points in 2D space
 - shapes e.g. circle, square made up of lines
 - properties for lines and shapes e.g. color, line thickness, line colour etc.
 - Since all these items will mostly work together, they should have the same methods of interaction even though they are different objects (classes)
-
- The composite DP is most suitable for this because it allows us to arrange these unrelated but cooperative objects together into a part-whole hierarchy such that all objects are handled the same way

Composite DP objects

- Base Component
 - The interface for all objects in the composition. A client program uses base component to work with the objects in the composition.
 - It can be an interface or an abstract class with some methods common to all the objects.
- Leaf
 - Defines the behaviour for basic elements in the composition.
 - Represents building blocks for the composition and implements base component.
 - It does not reference other components.
- Composite
 - It consists of leaf elements and implements the operations in base component.

- The composite pattern models tree structures that represent part-whole hierarchies with arbitrary depth and width
- With the DP, each individual object and their compositions are treated uniformly



How would it be implemented?

- Base component

```
public interface Shape {  
    public void draw(String fillColor);  
}
```

- This is the base from which composites and leaves will inherit from
- Usually base components are abstract classes or interface objects
- One (1) method prototype is shown in the example above which **MUST** be overridden by subclasses (because it's an interface)

- Leaves

```
public class Triangle implements Shape {  
    @Override  
    public void draw(String fillColor) {  
        System.out.println("Drawing triangle in "+fillColor);  
    }  
}
```

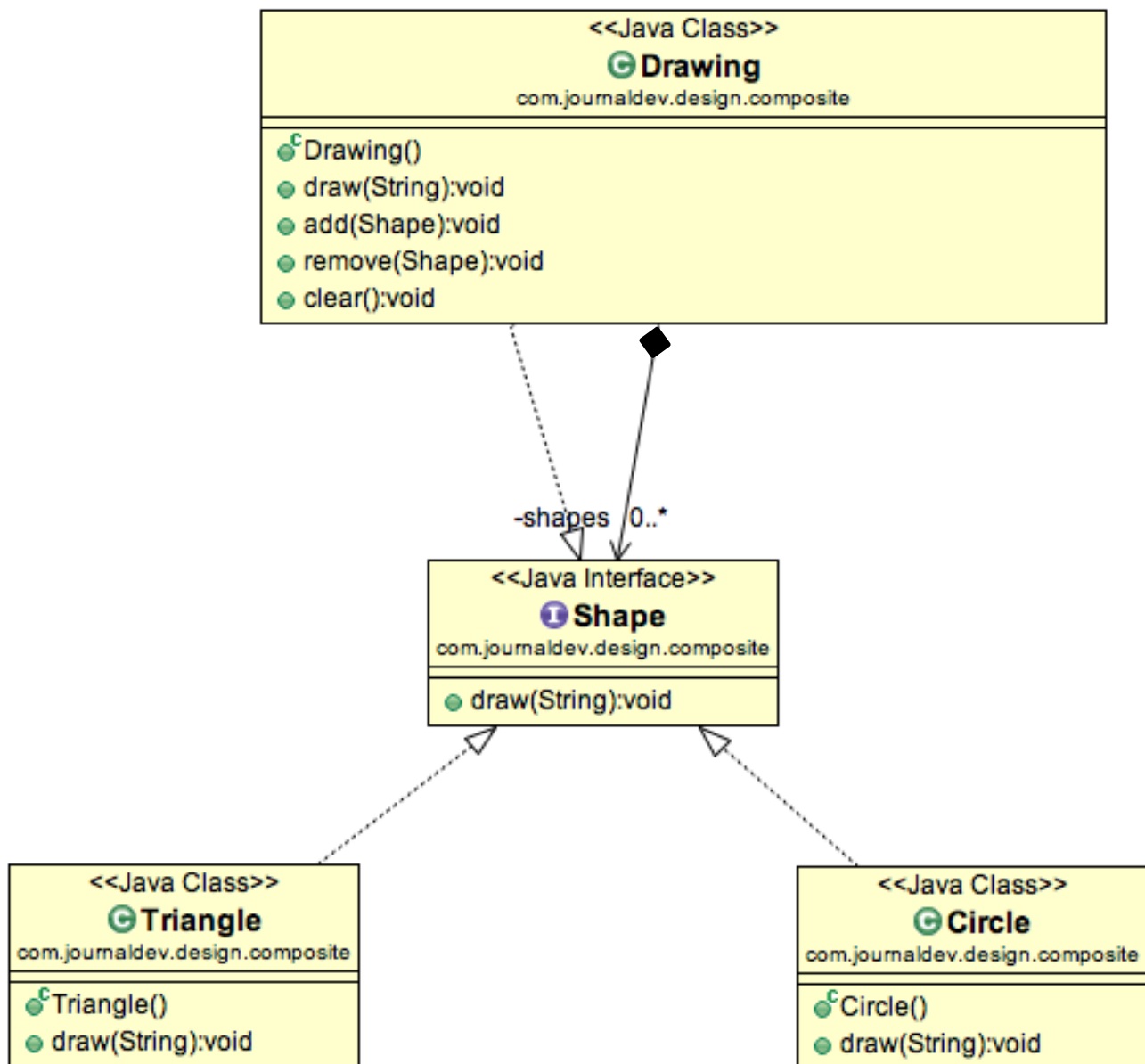
- A leaf implements the base component and these are the building block for the composites later.
- Note how it overrides the method prototypes from the superclass → in this case the method just prints to screen
- Another example

```
public class Circle implements Shape {  
    @Override  
    public void draw(String fillColor) {  
        System.out.println("Drawing circle in "+fillColor);  
    }  
}
```

- Composites

```
public class Drawing implements Shape{
    private List<Shape> shapes = new ArrayList<Shape>();
    @Override
    public void draw(String fillColor) {
        for(Shape sh : shapes){
            sh.draw(fillColor);
        }
    }
    public void add(Shape s){ shapes.add(s); }
    public void remove(Shape s){ shapes.remove(s); }
    public void clear(){ this.shapes.clear(); }
}
```

- Leaves may be too basic, so a *composite* object contains multiple leaf objects along with methods to manipulate the entire group
- Notice that composite also implements same base component and behaves similar to a leaf except that it can contain group of leaf elements.



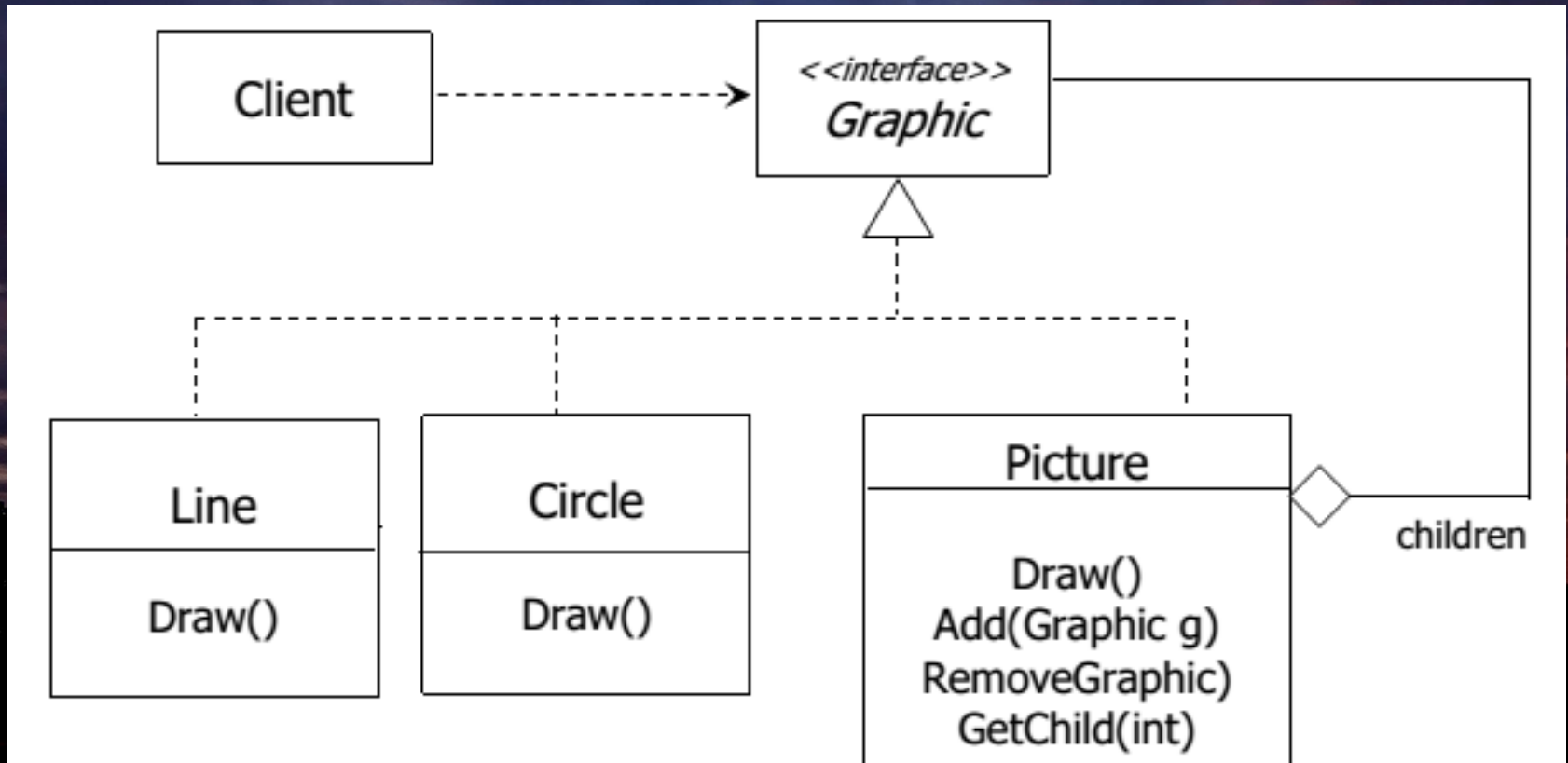
How is it used?

```
public class TestComposite {  
    public static void main(String[] args) {  
        Shape tri = new Triangle();  
        Drawing drawing = new Drawing();  
  
        drawing.add(new Triangle());  
        drawing.add(new Circle());  
  
        drawing.draw("Red");  
        tri.draw("Green");  
    }  
}
```

Leaf and composite
objects can be
declared like normal
classes

Both instances can be
controlled with the same
methods

- The Graphic class is an example that implements the **composite design pattern** with its hierarchy of drawable objects as both primitives (Line, Oval etc) and containers (Picture, Image etc)



Composite DP summary

Name	Composite
Intent	To treat individual objects and multiple, recursively-composed objects uniformly through a common interface
Applicability	Used when • you want to represent part-whole hierarchies of objects. • you want clients to be able to ignore the difference between compositions of objects and individual objects
Solution	Component interface specifies services that are shared amongst leaf and composite objects. A composite has an aggregation type association with components and implements each service by iterating each contained component. The leaf objects contain services that do actual work
Consequences	Defines class hierarchies consisting of primitive objects and composite objects Makes the client simple Makes it easy to add new types of components Can make design overly general

For more information on Composite

- <https://www.geeksforgeeks.org/composite-design-pattern-in-java/>
- Google “Composite Design Pattern”





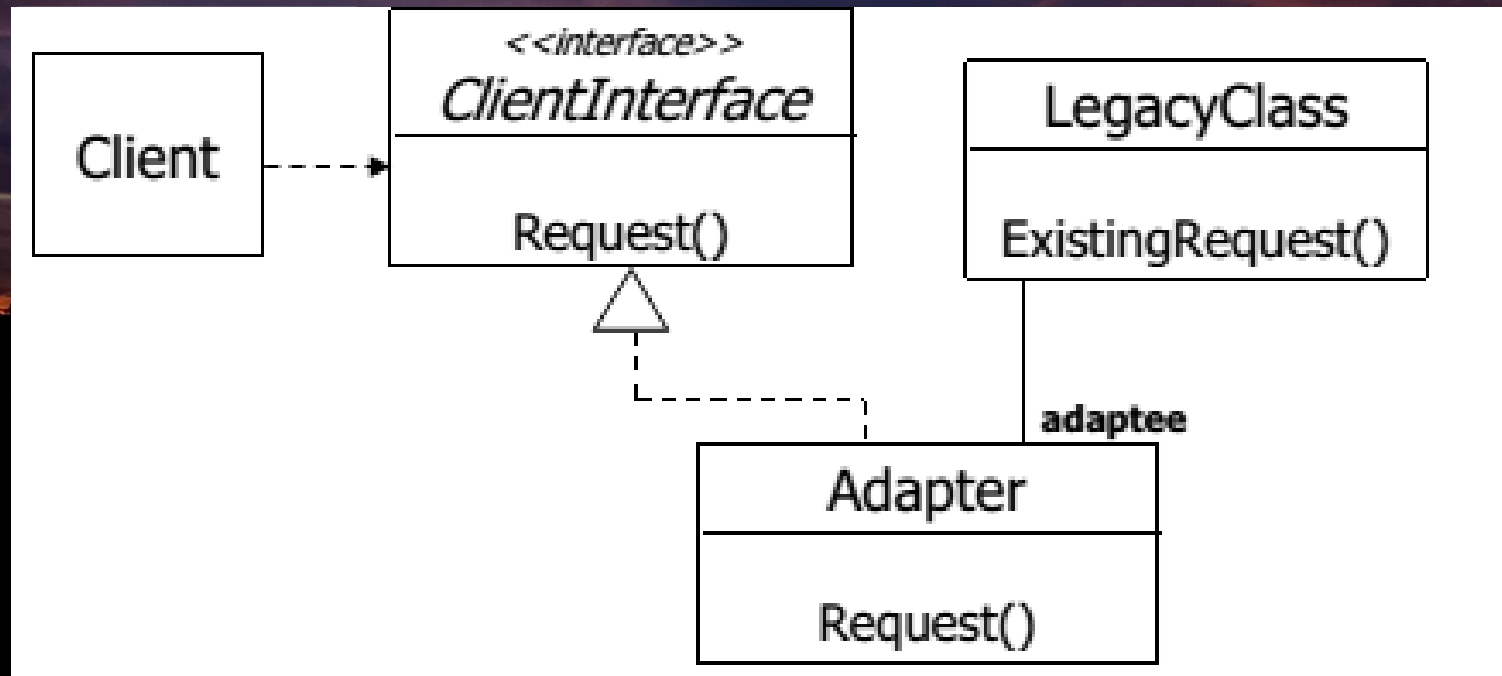
COMMON DESIGN PATTERNS

(Adapter - Structural)

Adapter DP scenario

- You are developing a program and require certain functionality but do not possess the skills/time to code it.
 - Luckily, you locate an existing software library that does the exact thing you require but it was written in a different language and only compiled binaries are provided.
 - Instead of rewriting or trying to understand the code, you decide to use the software library as is
-
- The adapter DP is suitable for this scenario because it basically allows two unrelated objects to work together.

- The adapter DP (also called mediator or wrapper DP) is used to
 - Convert the interface of a class into another interface clients expect (e.g. in terms of data type, parameters, language etc)
 - Wrap an existing working class with a new interface (e.g. conversion from 32-bit to 64-bit)
 - Match/translate an old component into a new system - applied to integrate existing applications (legacy applications, COTS, objects...) into newly develop ones



Sample Code

```
class Client {  
    ClientInterface inter = new Adapter();  
    public void aMethod( ) {  
        inter.Request()  
    }  
}
```

```
interface ClientInterface {  
    public Object Request( );  
}
```

```
class LegacyApp {  
    public Object existingRequest( ) {  
    }  
}
```

```
class Adapter implements ClientInterface {  
    private LegacyApp legApp = new ...  
    public Object Request( ) {  
        legApp.existingRequest( );  
    }  
}
```

adaptee

How would it be implemented?

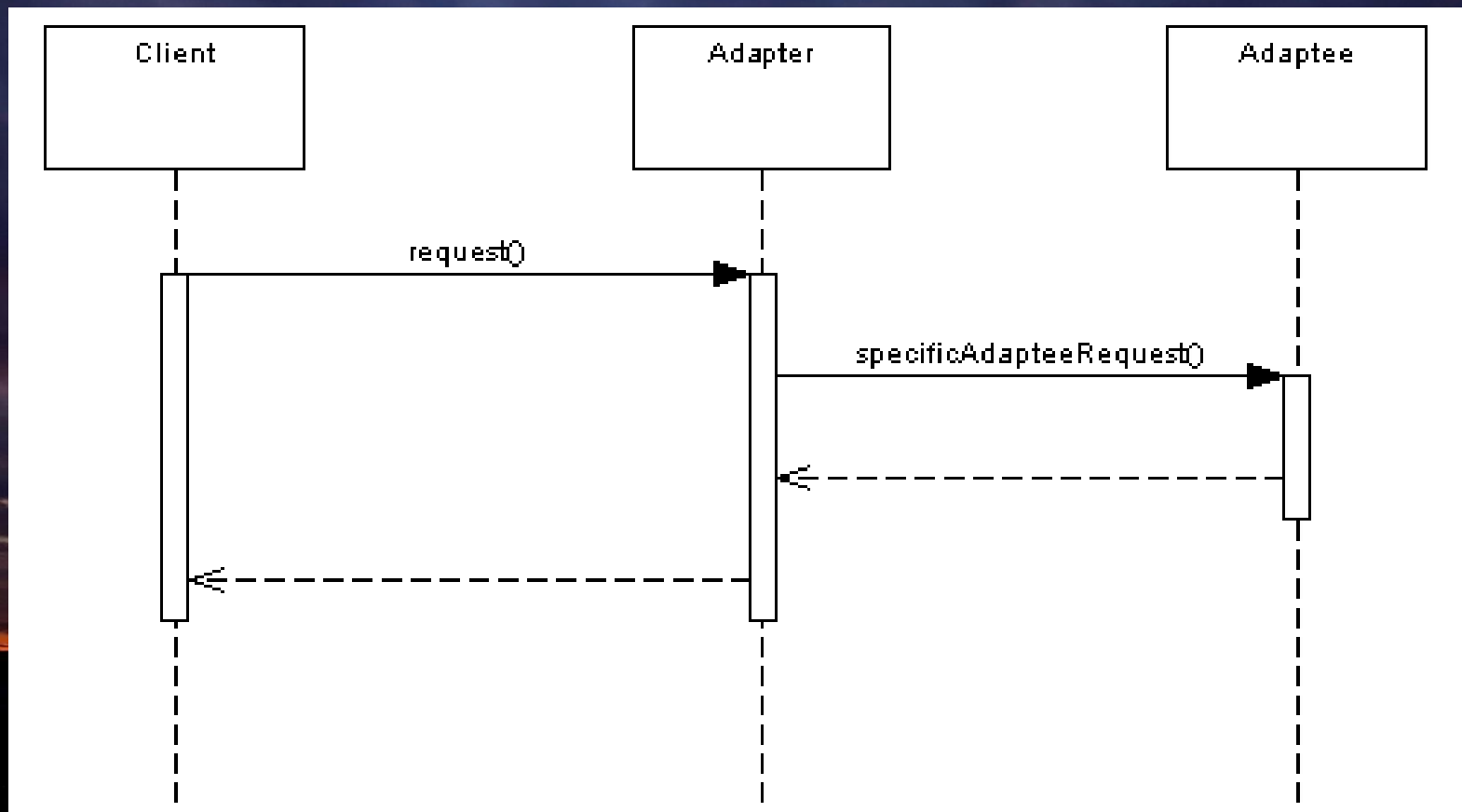
- Consider that you have a third party library that has a built in sort function but only works with List type objects

```
public class NumberSorter {  
    public List<Integer> sort(List<Integer> numbers) {  
        //sort and return code here  
        return new List<Integer>();  
    }  
}
```

- It has to work with a client program that generates integer arrays and expects sorted integer arrays in return too

```
int[] numbers = new int[]{34, 2, 4, 12, 1};  
    // numbers is passed and must be return as sorted array
```

- Due to the complexity of the client program and the closed-source nature of the third party library you cannot change them both



- Implement an adapter between the library and the client program

```
import java.util.Arrays;
import java.util.List;
```

translate integer primitive
array to List

```
public class SortListAdapter{
    public int[] sort(int[] numbers) {
        //convert the array to a List
        List<Integer> numberList = new ArrayList<Integer>();
        for (int i=0;i<numbers.length;i++){
            numberList.add(numbers[i])
        }

        //call the function
        NumberSorter sorter = new NumberSorter();
        numberList = sorter.sort(numberList);

        //convert the list back to an array and return
        int[] sortedNumbers = new int[numbers.length];
        for (int i=0;i<numberList.size();i++){
            sortedNumbers[i]=numberList.get(i);
        }
        return sortedNumbers;
    }
}
```

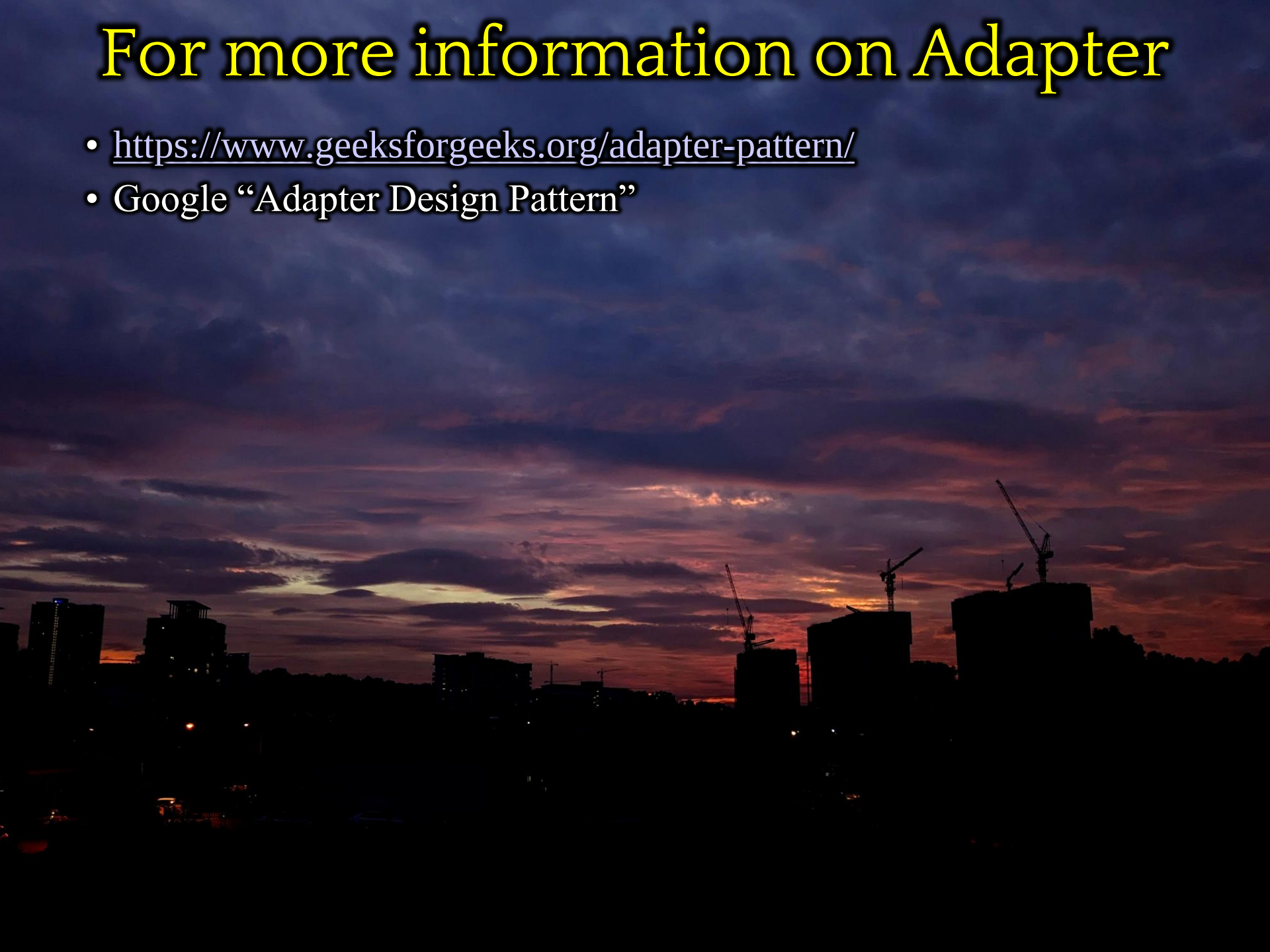
translate back to integer
array

Adapter DP summary

Name	Adapter
Intent	Convert interface of legacy class into an interface expected by a client so that the (new) client and legacy class can work together without changes
Applicability	Use the Adapter pattern when you want to • use an existing class but its interface doesn't match what you need. • create a reusable class that cooperates with unrelated /unforeseen classes • use several existing subclasses, but it's impractical to adapt their interface by subclassing every one.
Solution	Adapter class implements the ClientInterface interface expected by the client. The adapter delegates request from the client to the legacy class and performs necessary conversion to allow communication
Consequences	Client and legacy classes work together without modification to neither the client nor legacy class The adapter works with the legacy class and all of its subclasses A new adapter may be written for each subclass of the client

For more information on Adapter

- <https://www.geeksforgeeks.org/adapter-pattern/>
- Google “Adapter Design Pattern”





COMMON DESIGN PATTERNS

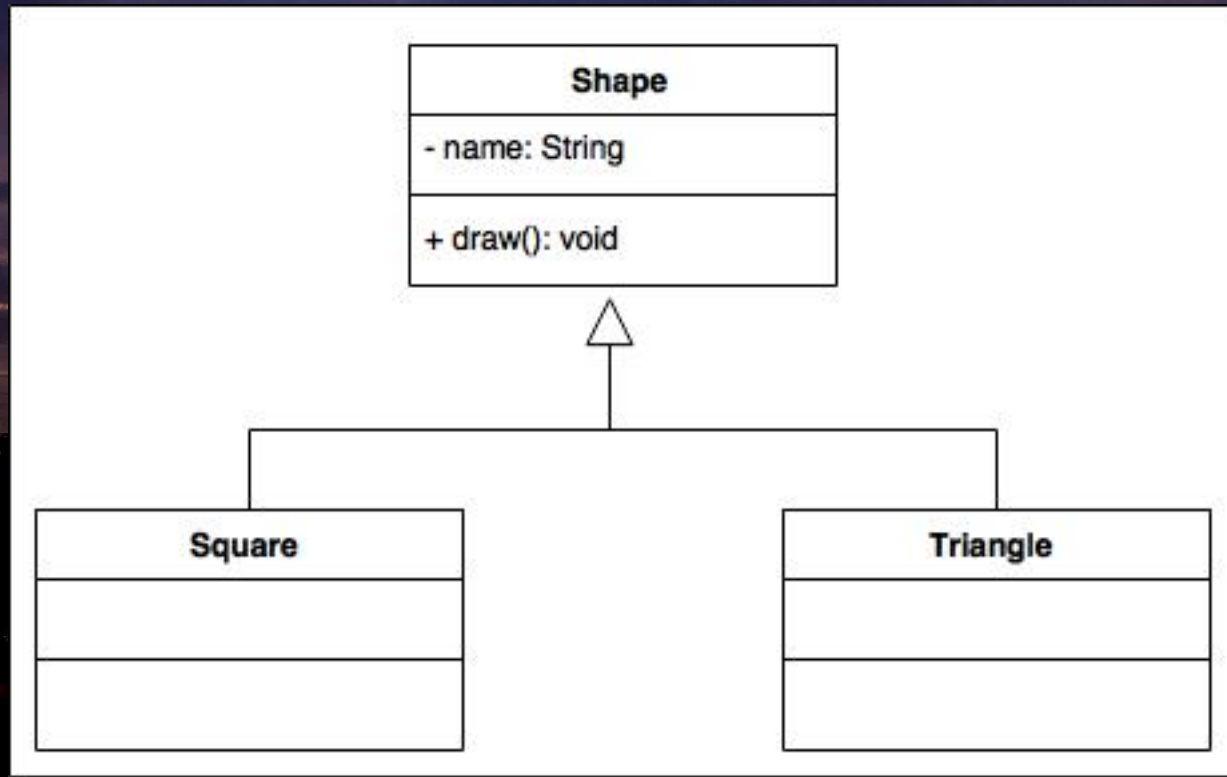
(Bridge - Structural)

Bridge DP scenario

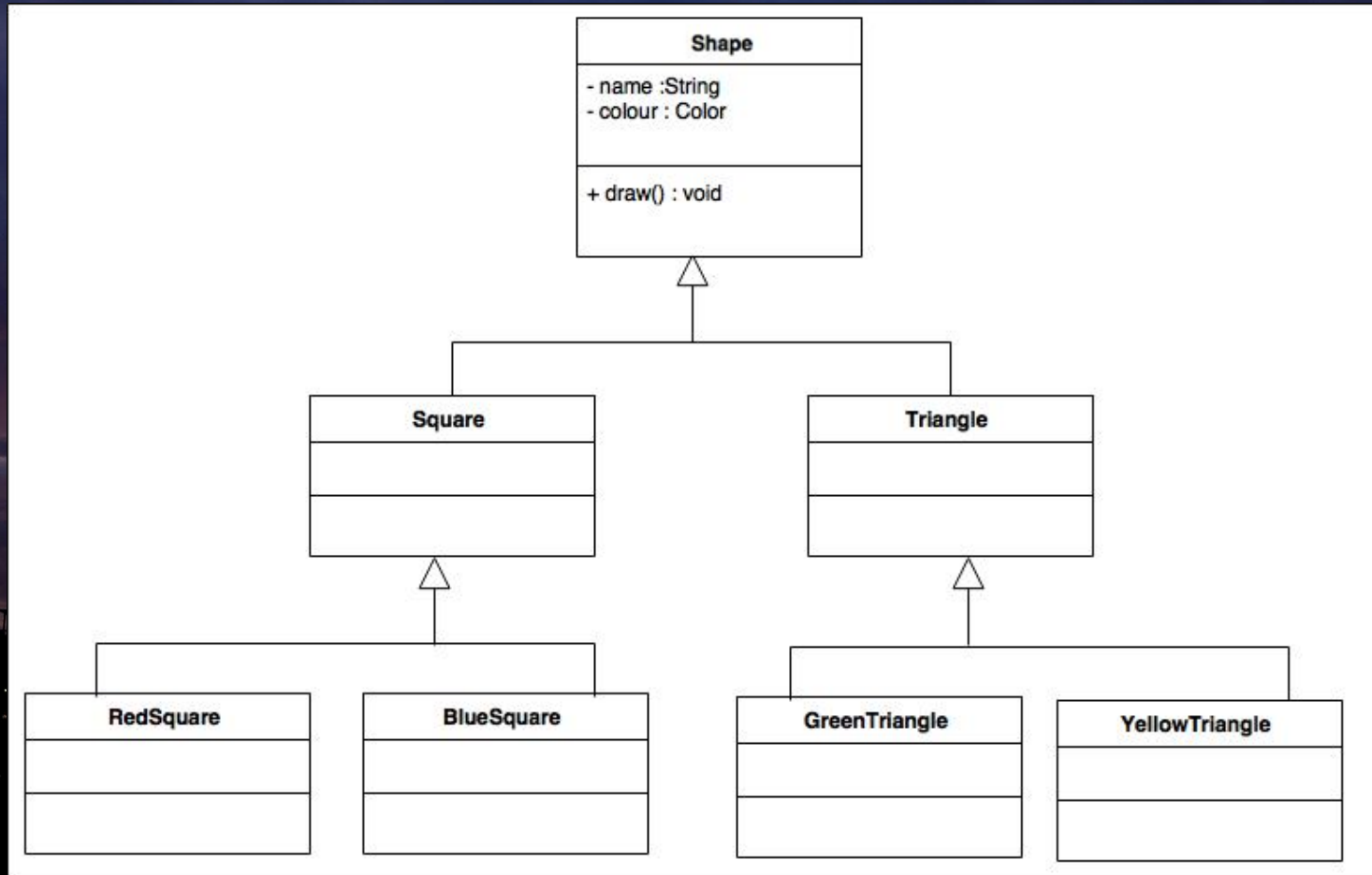
- You are designing a program for a client that draws shapes on the screen.
 - Later on the client demands that there can be different types of shapes and shapes with different colours
 - Unsatisfied with that, the client further adds that the shapes can be drawn solid, dashed or dotted lines.
 - Every time you meet the client, new demands are added
-
- The bridge DP is suitable for this scenario because it allows you to factor out common properties and make them become class implementations on their own

The OO way of implementation

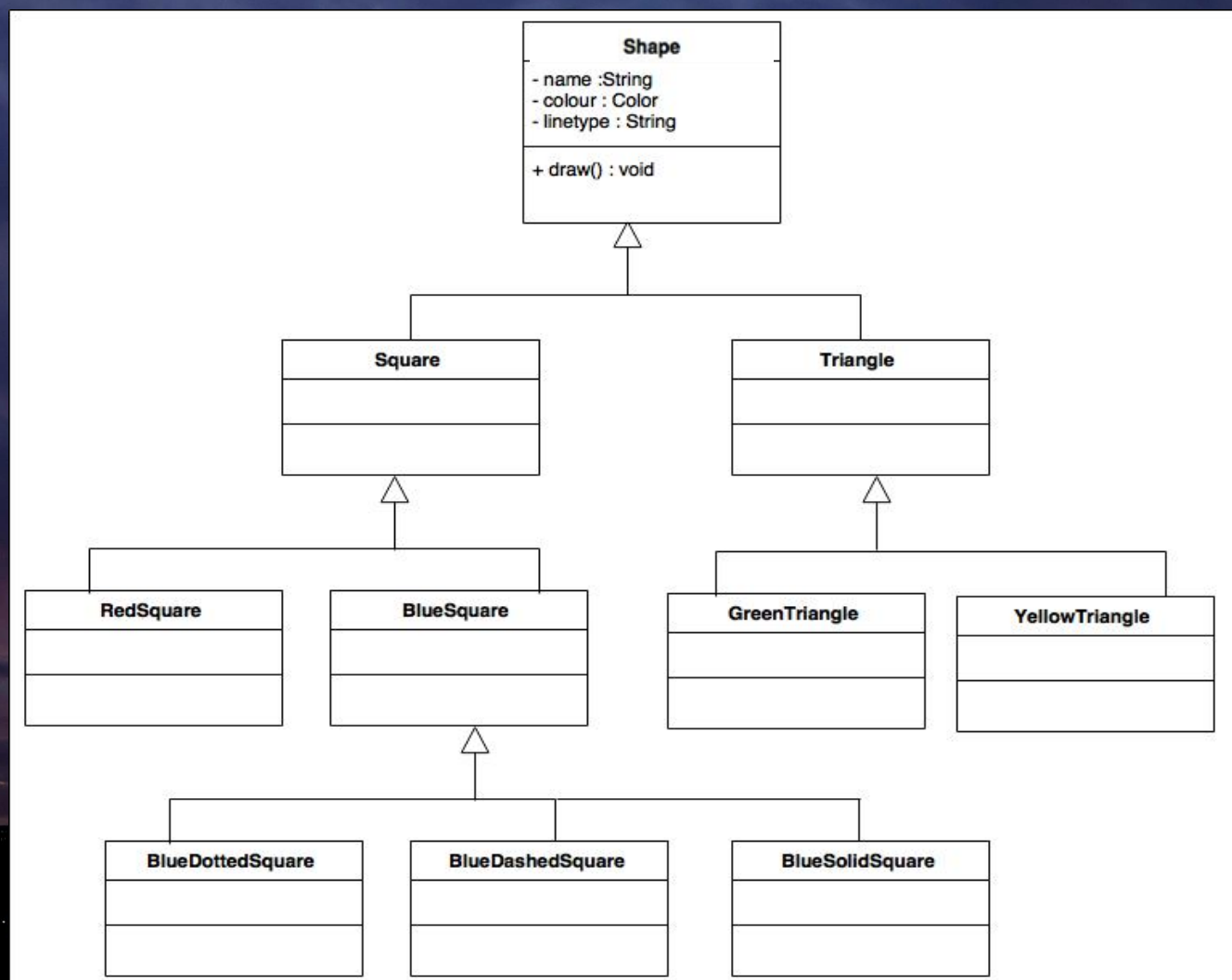
- You have learnt that OO model encourages inheritance and hierarchy of classes – the scenario shown before fits into this model nicely
- So, when the client wants the program to draw shapes, you could possibly start with an object hierarchy like this



- For client's request for shapes with colour, you can further specialise each shape with a particular colour type



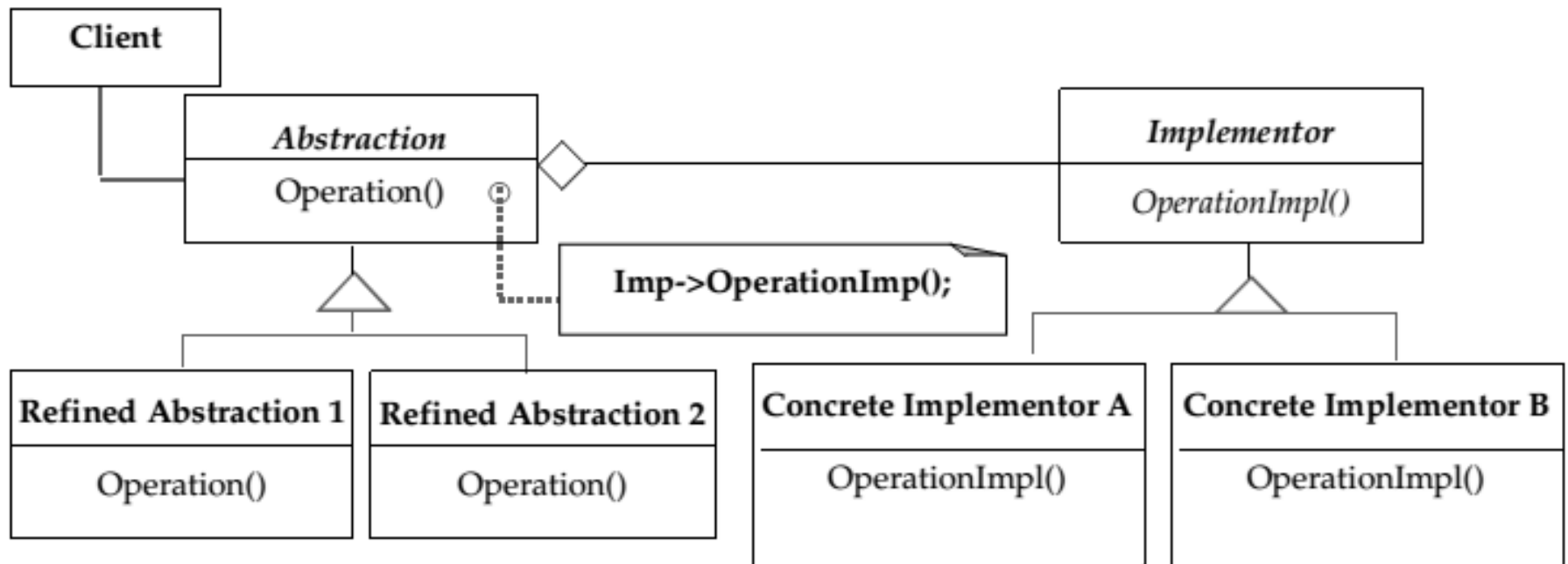
- Things begin to get messy once additional requirements start coming in from the client



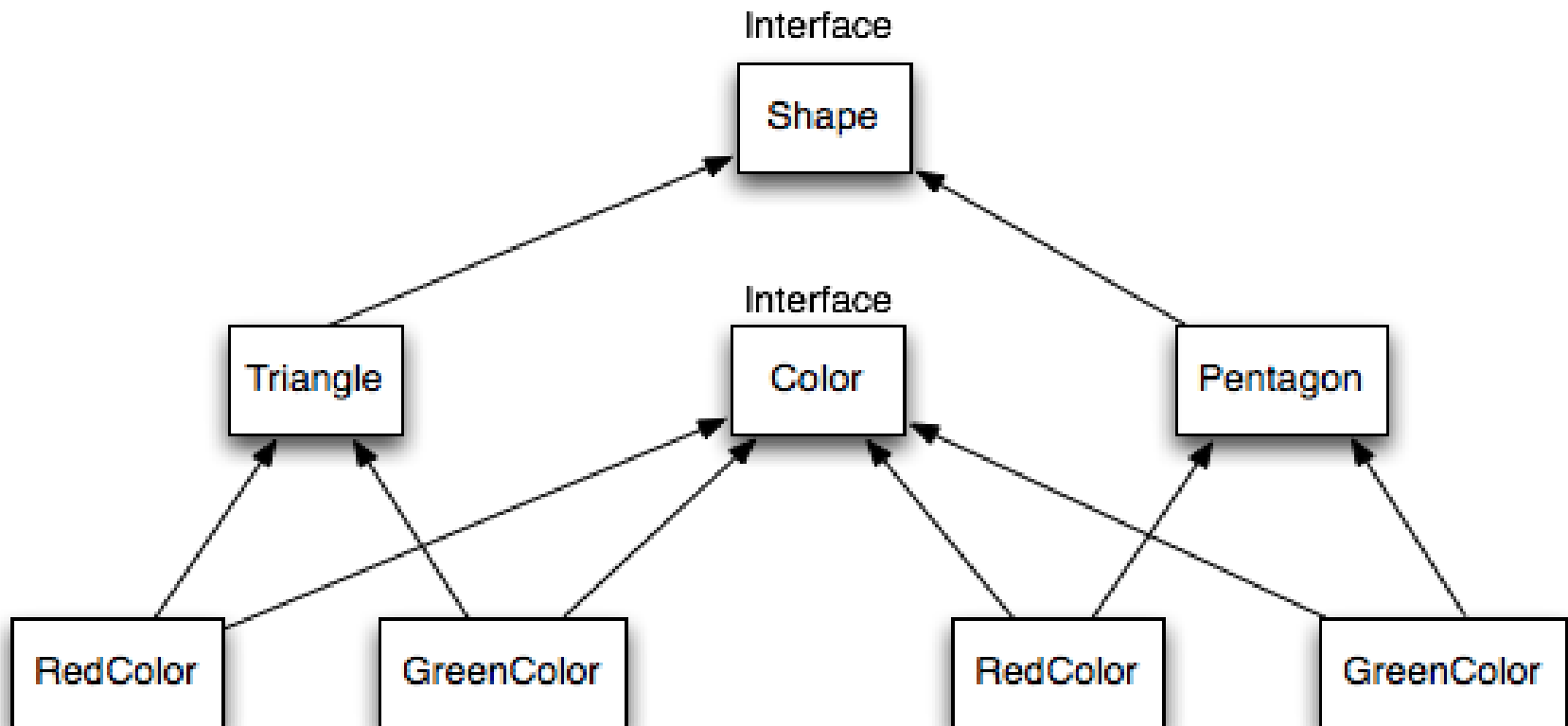
- Each object has to be specifically created during runtime here
- What happens if there are 20 shapes, 12 colours and 5 different line styles?

How to implement with DP?

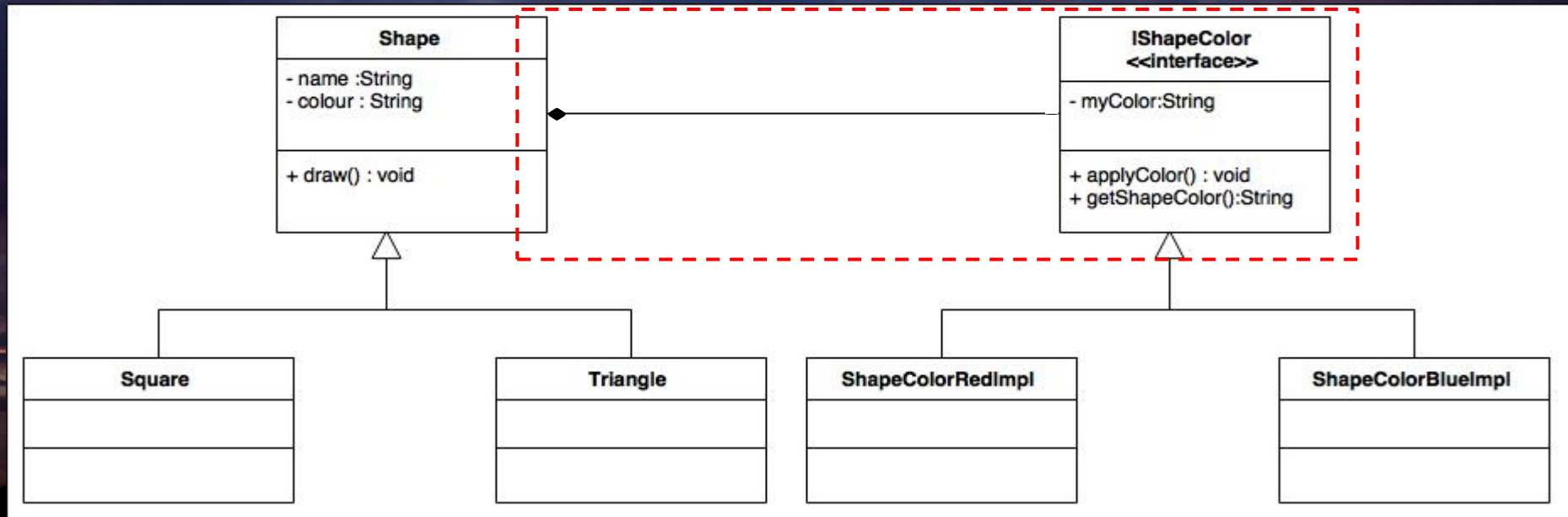
- This scenario is a simple example of using the *bridge DP* to create a better OO model which is easier to maintain
- Bridge DPs *decouples an abstraction from its implementations so that the two can vary independently*
- Allows different implementations of an interface which is decided upon dynamically



- Using bridge DP. the example before could have used refactoring to identify the colour of a shape as its own object



- Once refactored, a 'bridge' is created between the original Shape class and the new Color class
- If a new 'colour' is needed, then a subclass from ShapeColour interface object is created without affecting the original Shape



- The same process can be applied to 'bridge' the line style in Shape
- Now that each characteristic is no longer hard-coded, the objects can be configured during run-time

Bridge DP example in code

Abstraction

```
public abstract class Shape {  
  
    protected String name;  
    protected String color;  
  
    protected Shape() {  
        this.name = "no name";  
        this.color = "no color";  
    }  
  
    public void draw() {  
        System.out.println("I am " + this.name + " and my color is " + this.color);  
    }  
}
```

```
public class Circle extends Shape {  
  
    public Circle(IShapeColor shapeColor) {  
        super();  
        this.name = "Circle";  
        this.color = shapeColor.getShapeColor();  
    }  
}
```

Implementation

```
public interface IShapeColor {  
    public String getShapeColor();  
}
```

```
public class ShapeColorRedImpl implements IShapeColor {  
  
    private String myColor;  
  
    public ShapeColorRedImpl() {  
        this.myColor = "red";  
    }  
  
    @Override  
    public String getShapeColor() {  
        return this.myColor;  
    }  
}
```

Client

```
public class BridgePatternController {  
    public static void main(String[] args) {  
        Shape s1 = new Circle(new ShapeColorBlueImpl());  
        Shape s2 = new Rectangle(new ShapeColorRedImpl());  
        Shape s3 = new Triangle(new ShapeColorGreenImpl());  
  
        // It's very easy to draw circle or rectangle.  
        s1.draw();  
        s2.draw();  
        s3.draw();  
    }  
}
```

A new 'blue' IShapeColor is instantiated first. This object then is used to instantiate a new 'Circle' object – the constructor calls the getShapeColor method to obtain attribute value

This would 'draw' a blue circle, red rectangle and green triangle

Bridge DP summary

Name	Bridge
Motivation	To decouple an interface from an implementation so that implementations can be substituted – possibly during runtime
Applicability	Used when • Need to avoid a permanent binding between an abstraction and its implementation • Both abstractions and implementation should be extensible through subclassing • Need to isolate changes in implementations from clients • Need to split objects because of proliferation of classes
Solution	The Abstraction Class defines the interface visible to the client. The Implementor is an abstract class that defines the lower-level methods available to the Abstraction. An Abstraction instance maintains a reference to its corresponding Implementor instance. Both Abstraction and Implementor can be refined independently.
Consequences	Client is shielded from abstract and concrete implementations Interfaces and implementations can be handled and refined separately

For more information on Bridge

- <https://www.geeksforgeeks.org/bridge-design-pattern/>
- Google “Bridge Design Pattern”

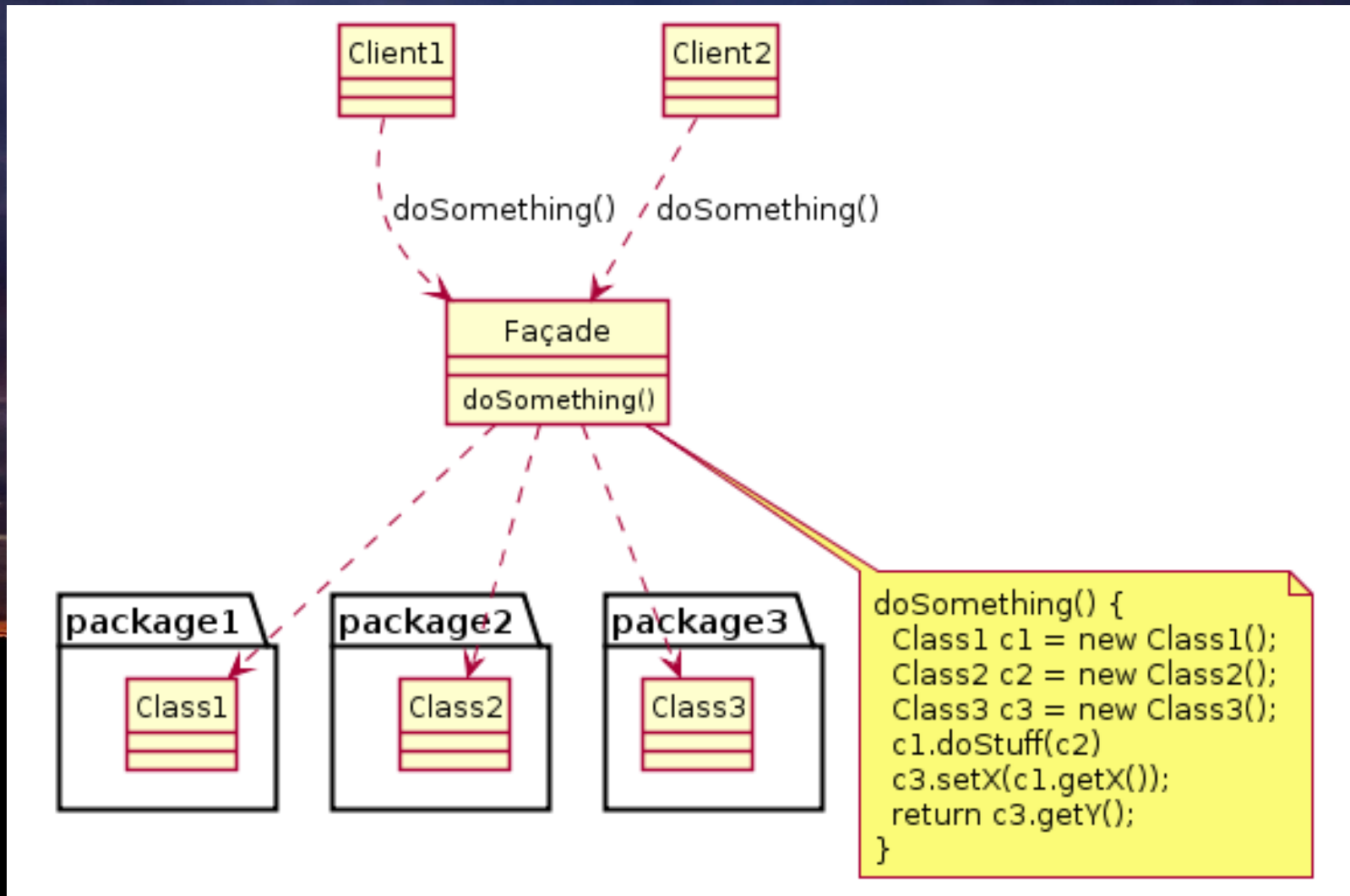


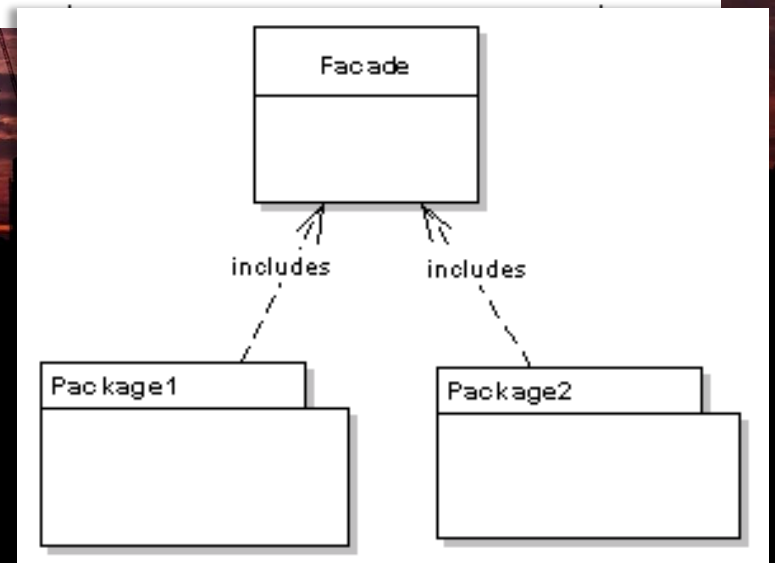
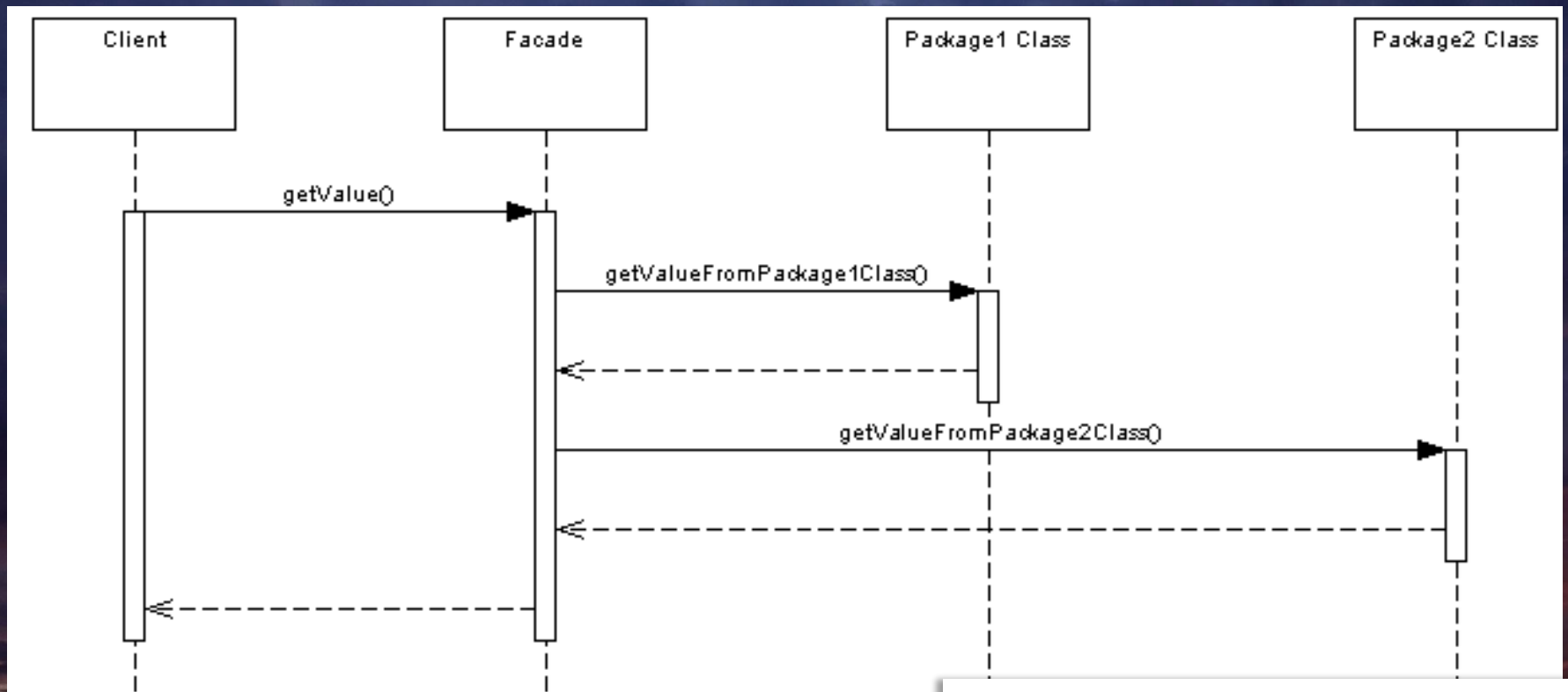
COMMON DESIGN PATTERNS
(Façade - Structural)

Façade DP scenario

- You have an application with set of individual interfaces to use a MySQL and Oracle database
 - It generates different types of reports, such as HTML report, PDF report etc. also using ready interfaces
 - A client program can make use of these interfaces to connect to the proper DB and generate required report
 - Maintenance of the interfaces becomes unwieldy when complexity and change demands increases
-
- The façade DP is suitable for this scenario because it wraps each individual interface in the system with one unified standard wrapper interface to help client applications

- Fairly similar to the adapter pattern, the Façade can be used to hide the inner workings of a third party library, or some legacy code





How is it implemented?

- Let's take a travel agent site for example, that allows you to book hotels and flights
- For each customer that visits the site, we assume that they will be looking for flight tickets and hotel accommodation → we need a flight booker and hotel booker object to handle these requests

```
public class HotelBooker {  
    public ArrayList<Hotel> getHotelNames(Date from, Date to) {  
        //returns hotels available in the particular date range  
    }  
}
```

```
public class FlightBooker{  
    public ArrayList<Flight> getFlights (Date from, Date to) {  
        //returns flights available in the particular date range  
    }  
}
```

- Without a design pattern, a client program (one that will use the hotel and flight Booker objects) would look something like this

```
public class Client {  
    private HotelBooker hotelBooker;  
    private FlightBooker flightBooker;  
    public static void main(String[] args)  
    {  
        ArrayList<Flight> flights = flightBooker.getFlights(from,to) ;  
        ArrayList<Hotel> hotels = hotelBooker.getHotels(from,to) ;  
        // process arrays of flights and hotels to get best deal  
    }  
}
```

- The client program has to create instances of each object and query separately imagine if there were 100 airlines and thousands of hotel chains, each with their own array and query

- Using the façade design pattern, the client program would look something like this and only access the façade interface thus hiding most of the implementation from the client program

```
public class TravelBookerFacade {  
    private HotelBooker hotelBooker;  
    private FlightBooker flightBooker;  
  
    public void getFlightsAndHotels(Date from, Date to) {  
        ArrayList<Flight> flights = flightBooker.getFlights(from, to);  
        ArrayList<Hotel> hotels = hotelBooker.getHotels(from, to);  
        //process and return  
    }  
}
```

```
public class Client {  
    public static void main(String[] args)  
    {  
        TravelFacade facade = new TravelFacade();  
        facade.getFlightsAndHotels(from, to);  
    }  
}
```

Façade DP summary

Name	Façade
Motivation	To provide a unified interface to a set of related objects in a subsystem
Applicability	Use façade when • Need to provide a simple interface to a complex system • Need to decouple a subsystem from its clients • Need to provide an interface to a software layer
Solution	A façade defines a high-level interface that makes subsystem objects easier to use → it abstracts out all the details The façade methods delegate client requests to the relevant subsystem classes Subsystem classes do not know about the façade class
Consequences	Clients limited to use only the façade class methods – hides implementation of subsystem! Promotes weak coupling between subsystem and clients (this is good) Simplifies porting systems to other platforms Façade does not prevent clients from accessing underlying classes!!!

For more information on Façade

- <https://www.baeldung.com/java-facade-pattern>
- Google “Façade design pattern”



Decorator – Structural

- The decorator pattern is used to extend or alter the functionality of objects at run-time by wrapping them in an object of a decorator class. This provides a flexible alternative to using inheritance to modify behaviour.
- <https://refactoring.guru/design-patterns/decorator>
- For more information, Google “Decorator design pattern”



Flyweight – Structural

- The flyweight pattern is used to reduce the memory and resource usage for complex models containing many hundreds, thousands or hundreds of thousands of similar objects.
- <https://refactoring.guru/design-patterns/flyweight>
- For more information, Google “Flyweight design pattern”



Proxy – Structural

- The proxy pattern is used to provide a surrogate or placeholder object, which references an underlying object. The proxy provides the same public interface as the underlying subject class, adding a level of indirection by accepting requests from a client object and passing these to the real subject object as necessary.
- <https://www.geeksforgeeks.org/proxy-design-pattern/>
- For more information, Google “Proxy Design Pattern”



Implementation differences in DPs

- Adapter makes things work *after* they're designed; Bridge makes them work *before* they are.
- The bridge DP is designed up-front to let the abstraction and the implementation vary independently. Adapter is retrofitted to make unrelated existing classes work together.
- Façade defines a new interface for objects, whereas Adapter reuses an old interface
- Composite lets clients treat individual objects and compositions of objects (usually of the same type) uniformly. Façade handles objects of different types.

When do you use the structural DPs?

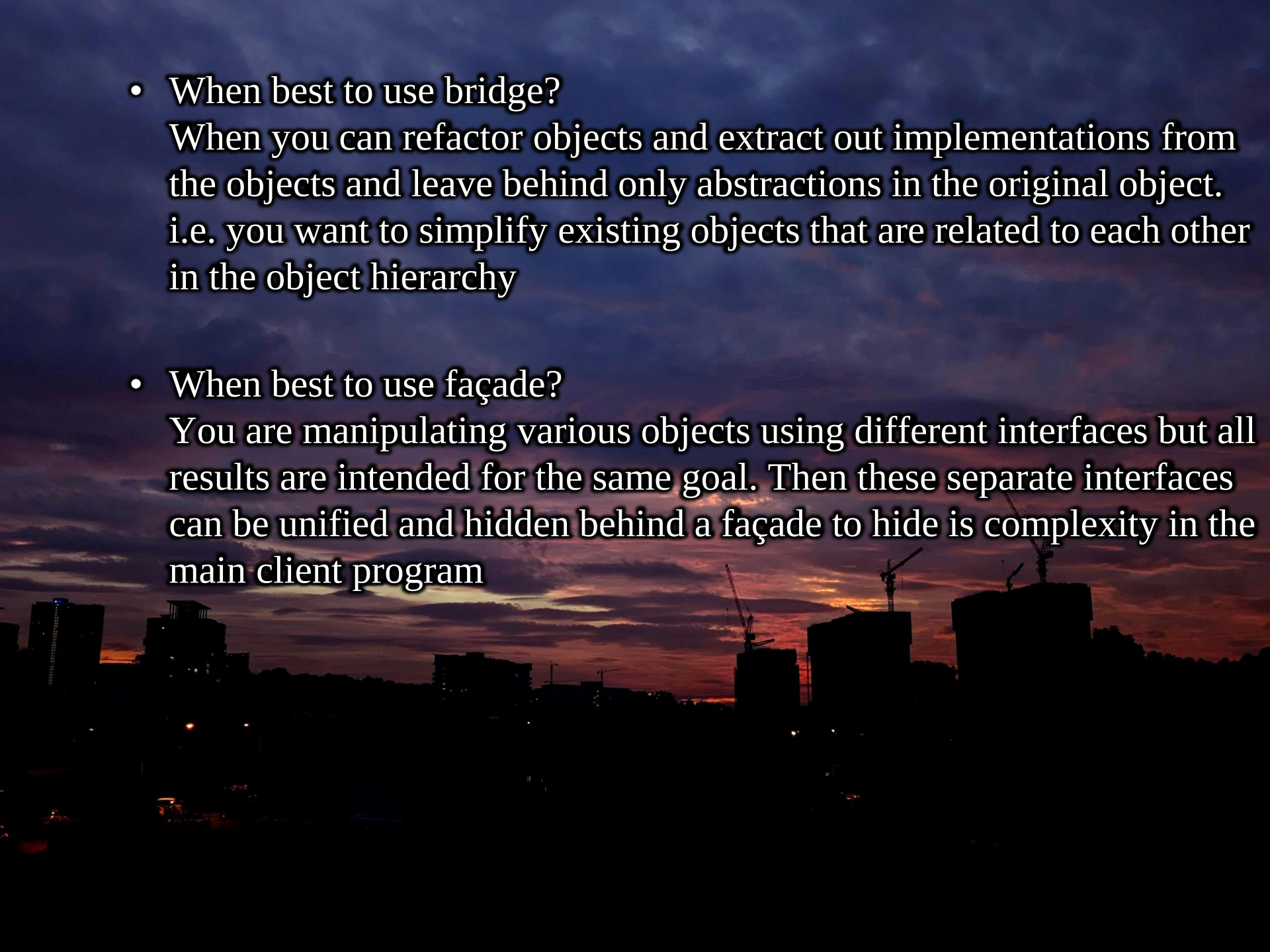
- The structural DPs are fairly similar between them and it may be difficult to determine when/where to implement the best DP (e.g. the bridge is basically variant of the adapter)
- When best to use composite?
You have many objects that can be handled separately and/or can be handled together as a single unit/group. The objects are usually similar in type and the method of manipulating the objects (whether singularly or as a group/composite) is the same
- When best to use adapter?
You have existing code/object/class/library that you cannot change and must make them work in another object. The adapter will help these different objects cooperate and work together by acting as a middleman/interpreter

- When best to use bridge?

When you can refactor objects and extract out implementations from the objects and leave behind only abstractions in the original object. i.e. you want to simplify existing objects that are related to each other in the object hierarchy

- When best to use façade?

You are manipulating various objects using different interfaces but all results are intended for the same goal. Then these separate interfaces can be unified and hidden behind a façade to hide its complexity in the main client program



Summary

- Design patterns are partial solutions to common problems such as separating an interface from a number of alternate implementations, wrapping around a set of legacy classes and protecting a caller from changes associated with specific platforms.
- DPs assist in creating better software design with methods that have been tried and tested

